University of Exeter

ECM2433: The C Family

Pointers to functions

Function pointers

You can also have a pointer to a function!

Consider these four functions:

```
int Plus(int a, int b) {  return a+b; }
int Minus(int a, int b) { return a-b; }
int Multiply(int a, int b) { return a*b; }
int Divide(int a, int b) {   return a/b; }
```

and now this function:

```
int doFunction (int a, int b, int (*myFunc)(int,int) )
{
     return (*myFunc)(a, b);
}
```

The third parameter is a pointer to a function that takes two int values as parameters and returns an int value.

Function pointers

```
int Plus     (int a, int b) { return a+b; }
int Minus    (int a, int b) { return a-b; }
int Multiply (int a, int b) { return a*b; }
int Divide   (int a, int b) { return a/b; }

int doFunction (int a, int b, int (* myFunc)(int,int) )
{
    return (*myFunc)(a, b);
}

int main ()
{
    int (*f)(int, int);
    int k;

    f = &Plus;
    k = (*f)(10, 6);

    printf("k = %d\n", k);

    printf("1+3 = %d\n", doFunction(1, 3, &Plus));
    printf("1*3 = %d\n", doFunction(1, 3, &Multiply));
    return 0;
}
```

Function pointers

```
int Plus     (int a, int b) { return a+b; }

int Minus    (int a, int b) { return a-b; }

int Multiply (int a, int b) { return a*b; }

int Divide   (int a, int b) { return a/b; }

int doFunction (int a, int b, int (* myFunc)(int,int) )
{
    return (*myFunc)(a, b);
}

int main ()
{
    int (*f)(int, int);
    int k;

    f = &Plus;
    k = (*f)(10, 6);

    printf("k = %d\n", k);

    printf("1+3 = %d\n", doFunction(1, 3, &Plus));
    printf("1*3 = %d\n", doFunction(1, 3, &Multiply));
    return 0;
}
```

```
k = 16
```

Function pointers

```
int Plus     (int a, int b) { return a+b; }
int Minus    (int a, int b) { return a-b; }
int Multiply (int a, int b) { return a*b; }
int Divide   (int a, int b) { return a/b; }

int doFunction (int a, int b, int (* myFunc)(int,int) )
{
    return (*myFunc)(a, b);
}

int main ()
{
    int (*f)(int, int);
    int k;

    f = &Plus;
    k = (*f)(10, 6);

    printf("k = %d\n", k);

    printf("1+3 = %d\n", doFunction(1, 3, &Plus));
    printf("1*3 = %d\n", doFunction(1, 3, &Multiply));
    return 0;
}
```

```
1+3 = 4
1*3 = 3
```

Function pointers

```
int Plus     (int a, int b) { return a+b; }

int Minus    (int a, int b) { return a-b; }

int Multiply (int a, int b) { return a*b; }

int Divide   (int a, int b) { return a/b; }

int doFunction (int a, int b, int (* myFunc)(int,int) )
{
    return myFunc(a, b);      // Dereferencing omitted
}

int main ()
{
    int (*f)(int, int);
    int k;

    f = Plus;                 // Address-of omitted
    k = f(10, 6);             // Dereferencing omitted
    printf("k = %d\n", k);

    printf("1+3 = %d\n", doFunction(1, 3, Plus));     // Address-of omitted
    printf("1*3 = %d\n", doFunction(1, 3, Multiply)); // Address-of omitted
    return 0;
}
```

Uses of function pointers

University of Exeter

Prototype for a sorting function[1], provided by stdlib.h:

```
void qsort (void *data,
            int   numElements,
            int   sizeOfElement,
            int   (*comparisonFunction)(void *, void *) );
```

[1]See https://sourceware.org/glibc/manual/latest/html_node/Searching-and-Sorting.html or “man qsort” for details

Function pointers

```
int CompareFunction(void *v1 , void *v2 )
{
   /* Return:
    * positive if *v1 is considered greater than *v2,
    * negative if *v1 is considered less than *v2
    * and zero if they are equal
    */
   int *i1 = (int *)v1;   // cast void pointers
   int *i2 = (int *)v2;
   return *i1 - *i2;
}

int iarray[] = {4, 6, 8, 3, 2, 1, 5, 9, 0};
int len = sizeof(iarray)/sizeof(int);    /* Number of elements in array *
qsort(iarray, len, sizeof(int), &CompareFunction);
```

Summary

Function pointers allow

- Dynamic calling of functions
- Functions to handle many data-types: e.g. qsort()

https://creator.nightcafe.studio/creation/9KV